

Aula prática 04 - Iluminação e Sombreamento

Introdução à Computação Gráfica
Prof. Davi Santos

- Download dos códigos
- Documentação dos OpenGL 2.1
- Livro de referência
- LINUX: compile e execute os códigos com `g++ hello.c -o hello -lGL -lGLU -lglut` $\&\&$
`./hello`

1 Fontes de luz - `light.c`

- Compile e execute o código `light.c` e observe a imagem renderizada;
- Modifique os parâmetros de reflexão ambiente, difusa e especular da luz e observe o resultado
 - Por padrão, os parâmetros da luz usados são os da figura 1;
 - Use como base o código 1 para definir novos valores para a luz 0.
 - O parâmetro difuso define o que nós entendemos como cor da luz. O parâmetro especular define a cor da reflexão daquela fonte de luz. O parâmetro ambiente define a contribuição residual daquela fonte de luz na cena.
- Troque a fonte de luz de direcional para posicional, e mude a cor da luz de branca para vermelha
 - O parâmetro `w` de posição da luz indica se a luz é pontual ou direcional. Com `w = 0` a luz é direcional e `(x, y, z)` é sua direção. Com `w = 1` a luz é pontual e `(x, y, z)` é sua posição.
- Adicione uma segunda fonte de luz do tipo lanterna (spotlight) e de cor verde à cena (dica: use o código 1). Coloque a luz do lado esquerdo do objeto (em relação a câmera).

Listing 1: Adicionando fontes de luz à cena.

```
// declara a cor (R,G,B,A) e posicao (x,y,z,w) da fonte de luz
GLfloat light1_ambient[] = { 0.2, 0.2, 0.2, 1.0 };
GLfloat light1_diffuse[] = { 1.0, 1.0, 1.0, 1.0 };
GLfloat light1_specular[] = { 1.0, 1.0, 1.0, 1.0 };
GLfloat light1_position[] = { -2.0, 2.0, 1.0, 1.0 };
GLfloat spot_direction[] = { -1.0, -1.0, 0.0 };
```

```

// atribui os valores de reflexao e posicao
glLightfv(GL_LIGHT1, GL_AMBIENT, light1_ambient);
glLightfv(GL_LIGHT1, GL_DIFFUSE, light1_diffuse);
glLightfv(GL_LIGHT1, GL_SPECULAR, light1_specular);
glLightfv(GL_LIGHT1, GL_POSITION, light1_position);

// atribui as atenuacoes
glLightf(GL_LIGHT1, GL_CONSTANT_ATTENUATION, 1.5);
glLightf(GL_LIGHT1, GL_LINEAR_ATTENUATION, 0.5);
glLightf(GL_LIGHT1, GL_QUADRATIC_ATTENUATION, 0.2);

// atribui as caracteristicas da luz tipo lanterna
glLightf(GL_LIGHT1, GL_SPOT_CUTOFF, 45.0);
glLightfv(GL_LIGHT1, GL_SPOT_DIRECTION, spot_direction);
glLightf(GL_LIGHT1, GL_SPOT_EXPONENT, 2.0);

// habilita a luz 1
glEnable(GL_LIGHT1);

```

Parameter Name	Default Value	Meaning
GL_AMBIENT	(0.0, 0.0, 0.0, 1.0)	ambient RGBA intensity of light
GL_DIFFUSE	(1.0, 1.0, 1.0, 1.0)	diffuse RGBA intensity of light
GL_SPECULAR	(1.0, 1.0, 1.0, 1.0)	specular RGBA intensity of light
GL_POSITION	(0.0, 0.0, 1.0, 0.0)	(x, y, z, w) position of light
GL_SPOT_DIRECTION	(0.0, 0.0, -1.0)	(x, y, z) direction of spotlight
GL_SPOT_EXPONENT	0.0	spotlight exponent
GL_SPOT_CUTOFF	180.0	spotlight cutoff angle
GL_CONSTANT_ATTENUATION	1.0	constant attenuation factor
GL_LINEAR_ATTENUATION	0.0	linear attenuation factor
GL_QUADRATIC_ATTENUATION	0.0	quadratic attenuation factor

Figure 1: Parâmetros da luz no OpenGL.

2 Movendo fontes de luz - movelight.c

No exemplo anterior, a posição da luz é estacionária na cena. Isto é feito declarando a posição da luz como a primeira matriz da pilha `modelview`. O código *movelight.c* mostra o caso de uma fonte de luz rotacionando em torno de um objeto.

- Compile e execute o código `movelight.c`. Mova a luz com botão esquerdo do mouse;
 - `spin` é uma variável global e controlada por um dispositivo de entrada. `display()` faz com que a cena seja redesenhada com a luz rotacionada com ângulo `spin` em torno de um toroide estacionário. Observe os dois pares de chamadas **`glPushMatrix()`** e **`glPopMatrix()`**, que são usadas para isolar as transformações de câmera e modelo, ambas ocorrendo na pilha `modelview`. Como no Exemplo 5-5 (ver livro) o ponto de vista permanece constante, a matriz atual é empurrada para baixo na pilha e, em seguida, a transformação da câmera desejada é carregada com **`gluLookAt()`**. A pilha de matrizes é empurrada novamente antes que a transformação do modelo **`glRotated()`** seja especificada. Em seguida, a posição da luz é definida no novo sistema de coordenadas rotacionado, de modo que a própria luz pareça estar girada em relação à sua posição anterior. (Lembre-se que a posição da luz é armazenada nas coordenadas da câmera, que são obtidas durante a transformação pela matriz de visualização). Depois que a matriz rotacionada é retirada da pilha, o toroide é desenhado.
- Troque a rotação da luz por uma translação e faça a luz passar por dentro do toroide (use o **`glTranslated()`**);
- Adicione um segundo toroide atrás do primeiro e faça a luz se mover junto com a câmera
 - Coloque a matriz de posição da luz como primeira transformação (mais embaixo) da pilha do `modelview`. **`glLightfv(GL_LIGHT0, GL_POSITION, position)`** com `position = (0.0, 0.0, 0.0, 1.0)`.
 - Mova a câmera. (agora quem translada é a câmera)

3 Sombreamento - light.c

- Modifique o **`glShadeModel(GL_SMOOTH)`** no `init()` por **`glShadeModel(GL_FLAT)`**. Observe o resultado;
- Renderize uma imagem suavizada usando o modelo de sombreamento FLAT
 - No **`glutSolidSphere(GLdouble radius, GLint slices, GLint stacks)`** os parâmetros `slices` e `stacks` referem-se ao número de cortes verticais e horizontais (em `z`) da esfera, semelhante a longitude e latitude. Aumente a ordem de grandeza do parâmetro **`stacks`** até obter uma imagem satisfatória. Continue aumentando os dois parâmetros (principalmente o **`stacks`**) e observe o tempo de renderização;

- Troque o sombreamento para SMOOTH e compare o tempo de renderização. Quantos **slices** e **stacks** são suficientes para uma imagem satisfatória com o SMOOTH?

4 Várias fontes de luz - light.c

- Adicione três fontes de luz pontuais, uma vermelha, uma verde e uma azul;
- Defina a posição das luzes de modo que a vermelha fique em cima da esfera, a azul fique na esquerda e a verde fique na direita (em relação a câmera);
- Defina uma tecla para ativar ou desativar as luzes;
 - Use `glDisable(LIGHTX)` para desativar a luz X;
- Faça as três luzes rotacionarem em torno da esfera, de modo que a luz vermelha rotacione em torno de x, a luz verde rotacione em torno de y e a luz azul rotacione em torno de z (a mesma tecla rotaciona as três).
 - Dica: use o `display()` e o `reshape()` do código `movelight.c`;
 - Dica: Use o `mouse()` do código `movelight.c`.

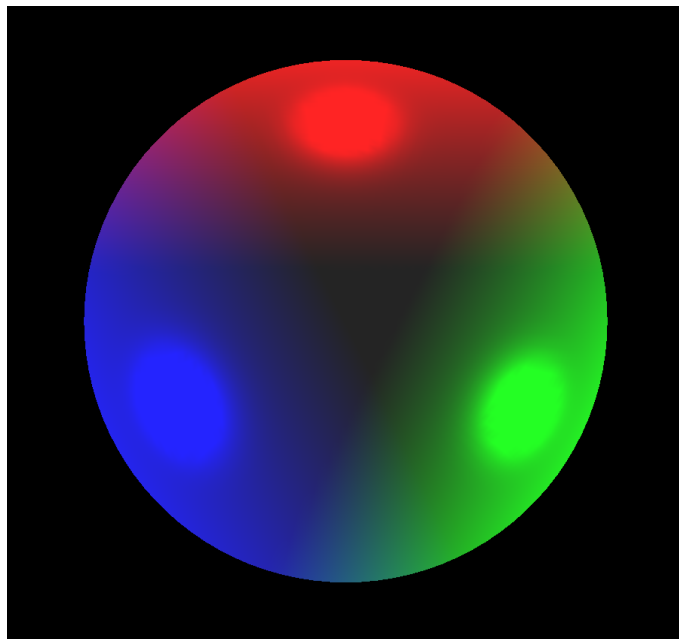


Figure 2: Resultado esperado.